Exercise 6-1 Practice routing

In this exercise, you'll review the essential skills for working with routing.

View and test the default route for the app

1. Open the Ch06Ex1RoutingPractice web app in the ex_starts directory.
2. Open the Program.cs file and view its code. Note that it includes the default route:

```
{controller=Home}/{action=Index}/{id?}
```

3. Open the HomeController class in the Controllers folder and view its code. Note that the Index() action method displays text that says "Home" and the Privacy() action method displays text that says "Privacy".
4. Run the app. This should start your browser and automatically call this URL:

```
https://localhost:<port_number>
```

   This should display text that says, "Home".

5. Enter URLs in the browser to display the text returned by the Index() and Privacy() methods. For example, try these URLs:

```
https://localhost:<port_number>/home
https://localhost:<port_number>/home/privacy
```

6. In the HomeController class, view the code for the Display() action method. Note that this method accepts a string parameter named id. Review the code that works with this parameter.
7. Run the app and and test the Display() method by entering these URLs:

```
/home/display
/home/display/123abc
```

   The first URL should display a message that indicates that the id hasn't been supplied, and the second URL should display a message that says, "ID: 123abc".

Add an action method that uses the default route

8. In the HomeController class, add the following action method. When you do, make sure to declare a parameter of the int type with a name of id and a default value of 0 like this:

```
public IActionResult Countdown(int id = 0)
{
    string contentString = "Counting down:\n";
    for(int i = id; i >= 0; i--)
    {
        contentString += i + "\n";
    }
    return Content(contentString);
}
```

9. Run the app and enter a URL that calls the Countdown() method like this:

```
/home/countdown
```

   This should display "Counting down:" followed by a 0 on the next line. That's because the URL omits the optional third segment of the default route. As a result, the method uses the default value of 0 for the id parameter.

10. Run the app and enter a URL that calls the Countdown() method like this:

    ```
    /home/countdown/10
    ```

    This should display a countdown that starts at 10 and counts down to 0 with each integer on its own line. That's because the URL included a value of 10 for the third segment.

Use attribute routing to customize the route for an action method

11. Add a Route attribute immediately above the Countdown() method like this:

    ```
    [Route("[action]/{id?}")]
    public IActionResult Countdown(int id = 0) {...}
    ```

12. Run the app and enter a URL that calls the Countdown() method like this:

    ```
    /countdown/10
    ```

    Note that you only have to type the name of the action (countdown) and the value of the id segment (10) in this URL, not the name of the controller (home).

13. Run the app and enter the previous URL that uses the default route to call the Countdown() method like this:

    ```
    home/countdown/10
    ```

    Now you should get an error message that the page you requested can't be found. That's because the routing attribute for the Countdown() method overrides the default route.

14. In the Countdown() method, change the name of the parameter from id to num. However, in the Route attribute, continue to use id as the name of the second segment.
15. Run the app and test the Countdown() method without an id segment. It should work correctly.
16. Test the Countdown() method with an id segment of 10 as before. This should only display the default value of 0. That's because the segment name in the Route attribute must match the parameter name in the method.
17. In the Route attribute, change the second segment from {id?} to {num?}.
18. Run the app and test the CountDown() method with a second segment of 10. This should display a countdown from 10 to 0.

Add more segments to a route

19. Edit the Countdown() method so it includes three parameters, and edit the Route attribute for this method so the segment names match the parameter names like this:

    ```
    [Route("[action]/{start}/{end?}/{message?}")]
    public IActionResult Countdown(int start, int end = 0,
        string message = "")
    ```

    Note that the first two segments are required but the third and fourth are optional.

20. Modify the code for the Countdown() method so it starts counting down at the start parameter, ends at the end parameter, and displays the message parameter when the countdown is done.
21. Run the app and enter a URL that calls the Countdown() method like this:

    ```
    /countdown/3/1/Liftoff!
    ```

    This should display a countdown like this:

    ```
    Counting down:
    3
    2
    1
    Liftoff!
    ```

22. Enter a URL that calls the Countdown() method like this:

    ```
    /countdown
    ```

    The browser should not be able to find the web page. That's because the second segment is now required. As a result, you must specify the start segment when you enter this URL.
23. Enter a URL that calls the Countdown() method like this:

    ```
    /countdown/5
    ```

    This should use the default parameters of the Countdown() method to display a countdown from 5 to 0 with no message.